Actividad III — Del incidente a la recuperación

Modelo B

Materia: Criptografía **NRC:** 1989 **Profesor:** Ricardo Luis Villanueva **Integrantes:** Joshua Lobo Rios y Santiago Vengoechea Noriega **Fecha:** Octubre de 2026

---

1. Introducción

Este trabajo simula, en un laboratorio aislado, el ciclo completo de un incidente de ransomware: desde la infección y el intercambio de una semilla maestra, pasando por el cifrado de los archivos de la víctima, hasta las dos formas de recuperarlos. Sobre esa base construimos además un atacante intermediario (MITM) que logra robar la semilla y, por último, una defensa con firmas digitales que impide ese robo.

Elegimos el **Modelo B**, en el que la víctima es quien genera la semilla maestra `sV` y se la entrega protegida al servidor. A lo largo del informe explicamos cómo quedó construida la aplicación, cómo se relaciona cada componente con los pasos del protocolo, y qué observamos al ejecutar las cuatro pruebas pedidas. Todas las pruebas se corrieron en contenedores Docker separados y, en los tres casos de recuperación, los archivos obtenidos resultaron **idénticos byte a byte** a los originales.

El reparto del trabajo fue el siguiente: Joshua se encargó del servidor y del núcleo criptográfico; Santiago, del cliente de la víctima, la capa de transporte y la investigación; el intermediario y las firmas digitales se diseñaron y programaron entre los dos, por tocar ambos extremos del protocolo a la vez.

2. El modelo elegido y los parámetros

En el Modelo B la víctima genera la semilla `sV` con la función del profesor `generar_semilla_maestra()`, la cifra con `OAEP.Enc(pkS, sV)` y la envía al servidor. El servidor la abre con su clave privada `skS` y la guarda asociada al identificador de la víctima (`idV`) junto con la clave pública `pkV`. La recuperación por pago es igual en ambos modelos: el servidor devuelve la semilla cifrada con la clave de la víctima, `OAEP.Enc(pkV, sV)`.

Fijamos los mismos parámetros criptográficos en todo el proyecto, tal como pide el enunciado:

- **RSA de 2048 bits con OAEP y SHA-256** para proteger la semilla maestra durante su transporte.
- **AES-256-GCM** para cifrar cada archivo, con un nonce de 12 bytes y una etiqueta de autenticación de 16 bytes.
- Una **clave distinta por archivo**, derivada con `Ki = HMAC-SHA256(sV, i)`, donde `i` es un contador escrito en 8 bytes (orden big-endian). El resultado de 32 bytes se usa directamente como clave AES-256. No se pide ninguna clave al servidor por archivo.
- Unos **datos asociados (AAD)** que amarran cada cifrado a su dueño y a su posición: `idV` (en UTF-8) concatenado con el contador `i` en 8 bytes big-endian.

La idea detrás de derivar las claves con HMAC es sencilla: basta guardar una sola semilla para poder regenerar, cuando haga falta, todas las claves de todos los archivos. Y la etiqueta de AES-GCM cumple un doble papel, porque además de descifrar permite comprobar que el contenido no fue

alterado y que la clave usada es la correcta.

3. Componentes de la aplicación

La aplicación se organiza en módulos con responsabilidades claras:

Componente	Archivo	Qué hace
Servidor	`servidor/servidor.py`	Registra a cada víctima, recibe y guarda `sV`, atiende la recuperación por pago y firma su clave pública
Núcleo criptográfico	`comun/crypto_common.py`	RSA-OAEP, derivación `Ki` con HMAC-SHA256 y AES-256-GCM con AAD
Transporte	`comun/transporte.py`	Envío de mensajes JSON sobre TCP con un prefijo de longitud de 4 bytes
Firmas	`comun/firmas.py`	Firma y verificación de claves públicas con RSA-PSS
Generador	`comun/generador_semillas.py`	Suministrado por el profesor, sin modificar
Víctima	`victima/victima.py`	Registro, generación y entrega de `sV`, cifrado y recuperación por pago
Intermediario	`mitm/mitm.py`	Proxy que intercepta la semilla y la descifra con la maestra robada
Investigación	`investigacion/investigar.py`	Recupera `sV` por fuerza bruta, sin el servidor

Los mensajes del protocolo viajan como objetos JSON. Los principales son: el registro de la víctima (`registro`, con `idV`, `pkv_n` y `pkv_e`); la respuesta del servidor (`registro_ok`, con `pks_n`, `pks_e` y la firma `pks_firma`); la entrega de la semilla (`entrega_semilla`, con la encapsulación OAEP en base64); y la solicitud y respuesta de recuperación por pago (`recuperar_pago` y `recuperacion`).

Qué se guarda en disco. El servidor conserva su par RSA de cifrado, su par RSA de firma y un registro `idV -> {pkV, sV, pagado}` en formato JSON, todo dentro de un volumen de Docker que sobrevive al reinicio. La víctima guarda únicamente su `idV` y su par RSA; nunca escribe en disco la semilla `sV`, las claves por archivo `Ki` ni la encapsulación OAEP original. Cada archivo cifrado se guarda como un JSON autocontenido (`cifrado_<i>.json`) que incluye `idV`, el contador `i`, el nombre original, el nonce, la etiqueta y el texto cifrado. Con esos datos y la semilla, cualquier archivo se puede reconstruir.

4. Relación con los pasos del protocolo

La tabla siguiente conecta cada paso de los diagramas del enunciado con el lugar del código donde ocurre:

Paso	Dónde ocurre en el código
1. Iniciar y registrar	`victima.comando_cifrar` llama a `registrar`; el servidor responde en `manejar_registro`
2. Establecer la maestra (B)	la víctima genera `sV` y la cifra con `encapsular_oaep`; el servidor la abre en `manejar_entrega_semilla`
3. Cifrar y guardar	la víctima deriva `Ki`, cifra con `cifrar_con_aad` y guarda con `escribir_cifrado`
4. Cerrar y reiniciar	el servidor recarga su par RSA y el registro del volumen; la víctima conserva solo `idV`, su RSA y los cifrados
5a. Recuperación por pago	`victima.comando_recuperar_pago` y `servidor.manejar_recuperacion_pago`
5b. Recuperación por investigación	`investigacion/investigar.py`
6. Descifrar y verificar	tanto la víctima como la investigación regeneran `Ki` y usan `descifrar_con_aad`, que verifica la etiqueta GCM

Qué adaptamos del material de referencia. Tomamos como punto de partida el Cuaderno Colab 1, en particular las secciones de RSA-OAEP (sección 7) y de la aplicación cliente-servidor (sección 8). De allí adaptamos las operaciones de encapsulación OAEP y el esquema de comunicación por sockets con framing de longitud. Hicimos dos cambios importantes: reemplazamos `secrets.token_bytes(32)` por la función `generar_semilla_maestra()` del profesor, que es la que debíamos usar, y añadimos soporte de datos asociados (AAD) a AES-GCM, llamando a `aes.update(aad)` antes de cifrar o verificar, para ligar cada archivo a su `idV` y a su contador.

5. Prueba 1 — Cifrado y pago

Objetivo. Mostrar que los archivos cifrados permanecen en disco tras cerrar y reiniciar los procesos, y que el pago permite recuperar la semilla y los originales.

Procedimiento. Levantamos el servidor en el proyecto de Docker `p1` y ejecutamos la víctima con el identificador `v-p1`. La víctima se registró, verificó la firma de la clave del servidor, generó la semilla, la entregó cifrada y cifró los dos archivos de prueba (uno de texto y uno binario de 1 MiB). Después cerramos y reiniciamos el servidor, y por último pedimos la recuperación por pago.

Resultado. Al reiniciar, el servidor mostró en su traza `Clave RSA del servidor cargada.` y `Clave RSA de firma cargada.`, lo que confirma que recargó sus claves desde el volumen en lugar de generar unas nuevas, y conservó el registro de `v-p1` con su semilla. La recuperación por pago devolvió `sV`, y con ella se descifraron ambos archivos. La comparación byte a byte dio idénticos:

```
binario.bin      IDENTICO
documento.txt    IDENTICO
```

La evidencia queda en `evidencias/p1_pago/` (traza del servidor, cifrados, recuperados y archivo de comparación).

6. Prueba 2 — Investigación

Objetivo. Con el servidor apagado y usando solo la información permitida, recuperar la misma semilla y comparar el costo estimado con el medido.

6.1 Por qué el ataque es posible

La función que genera la semilla hace, en esencia, lo siguiente:

```
configuracion = secrets.randbelow(9) + 16   # un entero entre 16 y 24
valor = secrets.randbelow(1 << configuracion)
return valor.to_bytes(32, "big")
```

Aunque la semilla ocupa 32 bytes, el valor que contiene siempre es menor que `2**configuracion`, y `configuracion` nunca pasa de 24. Por lo tanto, el valor es **siempre menor que 2^24**, sin importar qué número haya tocado en `configuracion`. Aquí está el detalle interesante: `configuracion` no agranda el conjunto de semillas posibles, porque el rango `[0, 2^16)` está contenido dentro de `[0, 2^24)`. Lo único que cambia `configuracion` es qué tan probable es cada valor, no cuáles valores pueden salir. En consecuencia, el conjunto completo de semillas posibles es exactamente `{0, 1, …, 2^24 - 1}`, es decir **16 777 216 valores**: apenas **24 bits de entropía real**, muy lejos de los 256 bits que debería tener una clave AES-256. Esa debilidad es la que hace práctico el ataque por fuerza bruta.

6.2 Cómo recuperamos la semilla

Recorremos todos los candidatos de `0` a `2^24 - 1`. Para cada candidato reconstruimos una semilla de 32 bytes, derivamos la clave del archivo que usamos como referencia e intentamos verificar su etiqueta de AES-GCM con el nonce y los datos asociados correctos. La propia verificación de la etiqueta es el criterio para reconocer la semilla: una etiqueta válida tiene una probabilidad de coincidir por azar de `2^-128`, tan pequeña que podemos dar la semilla por correcta en cuanto una verificación pasa. No hace falta ningún otro indicio.

Añadimos dos mejoras. Primero, usamos como archivo de referencia el **cifrado más pequeño** (no necesariamente el archivo `i = 0`), porque verificar la etiqueta cuesta en proporción al tamaño del texto cifrado; cualquier archivo sirve, ya que su clave y sus datos asociados dependen solo de su propio contador. Segundo, repartimos el rango de búsqueda entre varios procesos con `multiprocessing`. Durante el ataque solo se usó información permitida —el código de la aplicación y del generador, y los archivos cifrados con sus metadatos públicos— y nunca el registro del servidor, las claves privadas ni la semilla guardada durante el cifrado.

6.3 Mediciones

Métrica	Valor
Entorno	Windows 11, Python 3.12.10, 16 procesos
Espacio de búsqueda	2^24 = 16 777 216 candidatos
Archivo de referencia	el de 41 bytes de texto cifrado
Semilla encontrada	430 775 (0x0692b7)
Candidatos probados	266 936

Métrica	Valor
Tiempo en recuperar `sV`	8.454 s (unos 31 574 candidatos por segundo)
Tiempo en descifrar los archivos	0.031 s

6.4 Estimación frente a lo observado

En teoría, el peor caso requiere probar los 2^24 candidatos, y en promedio la mitad, alrededor de 2^23 ≈ 8.4 millones, porque el ataque se detiene en cuanto acierta. En nuestra ejecución la semilla resultó ser 430 775, un valor bajo, así que bastaron unos 267 mil candidatos, bastante por debajo del promedio. Esto es esperable: el número de intentos depende de dónde caiga la semilla concreta de cada ejecución. A la velocidad que medimos, el peor caso rondaría los seis minutos, lo que coincide con la idea de que el ataque es "cuestión de minutos" para un espacio de 24 bits.

Conviene separar dos costos. Recuperar la semilla tomó **8.454 segundos**, mientras que descifrar los dos archivos una vez conocida la semilla tomó apenas **0.031 segundos**. El costo está dominado casi por completo por la búsqueda de la maestra. Como la semilla se busca una sola vez y después el descifrado crece de forma lineal con la cantidad de archivos, mientras más archivos tenga la víctima, más se amortiza el esfuerzo de la fuerza bruta: el costo caro se paga una vez y se aprovecha para todos.

La comparación final confirmó que los archivos recuperados por investigación son idénticos a los originales. La evidencia está en `evidencias/p2_investigacion/` (incluido `mediciones.csv`).

7. Prueba 3 — Intermediario (MITM)

Objetivo. Que un tercero intercepte la semilla durante su entrega, deje que la víctima termine su cifrado sin darse cuenta, y luego recupere los archivos sin pagar.

Procedimiento. Configuramos la víctima para conectarse al intermediario (puerto 5001) y a este para conectarse con el servidor, de modo que todos los mensajes pasen por él. En esta prueba la víctima acepta las claves sin verificar firmas, como pide el paso M2. El ataque, paso a paso:

- **M2 (sustitución).** El intermediario reenvió al servidor la clave de la víctima sin cambios, pero a la víctima le entregó su propia clave `pkM` en lugar de la del servidor `pkS`, guardando la `pkS` verdadera para más adelante.
- **M3 (intercepción).** La víctima cifró la semilla con `pkM` creyendo que era la del servidor. El intermediario la abrió con su clave privada `skM`, se quedó con la semilla y la volvió a cifrar con la `pkS` real.
- **M4 (reenvío).** El servidor recibió la entrega como si viniera directo de la víctima, la abrió con `skS` y confirmó. Esa confirmación se reenvió a la víctima, que no notó nada raro.
- **M5 (cifrado).** La víctima derivó sus claves y guardó los archivos cifrados.
- **M6 (recuperación sin pago).** Con la semilla interceptada y el servidor apagado, el intermediario regeneró las claves y descifró las copias.

La traza del intermediario muestra con claridad la sustitución, la apertura y el reenvío:

```
M2: clave pública sustituida (pkS -> pkM) para v-p3
M3: maestra interceptada de v-p3
    sV = 0000000000000000000000000000000000000000000000000000000001a6e21
```

```
M3: maestra re-cifrada con la pkS real y reenviada.
```

Por qué RSA-OAEP no evitó el robo. OAEP garantiza que un mensaje cifrado solo pueda abrirlo quien tenga la clave privada correspondiente, pero no dice nada sobre a quién pertenece realmente la clave pública que se usó. Como la víctima cifró con `pkM` sin comprobar de quién era, el intermediario —dueño legítimo de `skM`— pudo abrir la entrega sin ninguna dificultad. En otras palabras, OAEP protege el transporte del secreto, pero no la autenticidad de la clave pública; ese es justamente el hueco que explota el MITM. Los archivos recuperados por el intermediario resultaron idénticos a los originales (evidencia en `evidencias/p3_mitm/`).

8. Prueba 4 — Firmas digitales

Objetivo. Impedir la sustitución de claves del paso anterior, de modo que la víctima detecte el engaño antes de exponer la semilla.

8.1 Diseño de la defensa

- **Esquema de firma.** Usamos RSA-PSS con SHA-256, con un **par de claves de firma del servidor independiente** de su par de cifrado. El servidor firma su clave pública y la víctima la verifica. Separar firma y cifrado evita mezclar las dos funciones.
- **Qué se firma.** Se firma una representación canónica que incluye el protocolo, el rol ("servidor"), el `idV` y los parámetros `(n, e)` de la clave pública de cifrado. Así la firma ata la clave recibida tanto a la identidad del servidor como al `idV` de la víctima.
- **De dónde sale la confianza.** La víctima obtiene la clave de verificación del servidor **por un canal aparte**, antes del intercambio: copiamos el archivo `clave_firma_publica.pem` del servidor al entorno de la víctima. La víctima verifica siempre contra esa clave de confianza e **ignora** cualquier clave que llegue por el mismo canal del intercambio.
- **Cuándo se verifica.** La firma se comprueba **antes** de usar la clave para cifrar la semilla. Si la verificación falla, la víctima detiene el intercambio y **no** envía la maestra; nunca continúa con una simple advertencia.

8.2 Resultados

Probamos los tres casos que pide el enunciado:

Caso	Identificador	Vía	Resultado
1. Sustitución de clave	`v-p4-mitm`	Intermediario (5001)	Rechazado; no se envía la semilla
2. Firma propia del intermediario	`v-p4-firma-mitm`	Intermediario con firma propia (5001)	Rechazado; no se envía la semilla
3. Intercambio legítimo	`v-p4-legitima`	Servidor directo (5000)	Completo

La traza del servidor confirma el desenlace: las dos víctimas atacadas quedaron **registradas pero sin semilla**, mientras que la del intercambio legítimo muestra `Semilla maestra recibida y guardada para: v-p4-legitima`. La evidencia está en `evidencias/p4_firmas/` (traza del servidor y trazas del intermediario de cada caso).

8.3 Por qué se rechaza cada ataque

En el **caso 1**, el intermediario cambia los parámetros de la clave por los de `pkM` pero deja la firma original del servidor, que fue hecha sobre la clave verdadera `pkS`. Al verificar esa firma contra la clave de confianza, los datos firmados no coinciden con la clave recibida y la verificación falla.

En el **caso 2**, el intermediario firma `pkM` con su propia clave de firma e incluso adjunta su clave de verificación. Aun así la víctima lo rechaza, porque no verifica con la clave que entrega el atacante, sino con la del servidor real que ya tenía de antemano; la firma del intermediario no valida contra esa clave.

De aquí sale la conclusión más importante de la prueba: **confiar únicamente en la firma recibida no basta**. Si la víctima aceptara la clave de verificación que viene en el mismo intercambio, el intermediario podría reemplazar a la vez la clave de cifrado y la clave que verifica la firma, y pasar la comprobación. La seguridad no viene de la firma en sí, sino de **anclar la confianza** en una clave de verificación legítima conocida de antemano por un canal independiente. Respecto a RSA-OAEP, las firmas añaden algo que OAEP no daba: la **autenticación del origen** de la clave pública, que es exactamente lo que cerró el hueco aprovechado en la Prueba 3.

8.4 Diagrama adaptado del Modelo B con la defensa

El ciclo de los pasos 1 a 6 se mantiene; la defensa agrega la firma en el mensaje `registro_ok` y una verificación en la víctima antes del paso 2. El flujo queda así:

```
Víctima                                    Servidor
   |  1. registro: idV, pkV            ->    |
   |                                         |  firma pkS con su clave de firma
   |  1. registro_ok: pkS, firma(pkS)  <-    |
   |                                         |
   |  Verificar firma(pkS) con la            |
   |  clave de confianza pre-compartida      |
   |     - si NO valida: DETENER (no se      |
   |       envía la semilla)  ---------------X  (punto de rechazo)
   |     - si valida: continuar              |
   |                                         |
   |  2. entrega_semilla: OAEP.Enc(pkS, sV) ->   abre con skS y guarda sV
   |  2. entrega_ok                    <-    |
   |  3. cifrar archivos (Ki, AES-GCM)       |
   |  4. cerrar y reiniciar                  |
   |  5a. recuperar_pago / 6. descifrar      |
```

> Sugerencia: pueden reemplazar este esquema de texto por una figura dibujada (por ejemplo en draw.io o PowerPoint) y pegarla aquí antes de exportar a PDF.

9. Comparación final de archivos recuperados

En las tres vías de recuperación los archivos obtenidos coinciden exactamente con los originales reservados, verificado con SHA-256:

Prueba	binario.bin (1 MiB)	documento.txt
1. Pago	IDÉNTICO	IDÉNTICO
2. Investigación	IDÉNTICO	IDÉNTICO
3. MITM (M6)	IDÉNTICO	IDÉNTICO

Los archivos `comparacion.txt` de cada carpeta de evidencias respaldan esta tabla.

10. Conclusiones

El ciclo completo del Modelo B funciona de principio a fin y, lo más importante, sobrevive al reinicio: el servidor recupera su estado desde el disco y la víctima puede recuperar sus archivos sin haber guardado la semilla. Esto confirma que el diseño no depende de información que viva solo en memoria.

La investigación dejó clara una lección de criptografía práctica: un algoritmo fuerte como AES-256 no sirve de nada si la clave proviene de una fuente con poca entropía. El generador, al limitar las semillas a 24 bits, convirtió una clave teóricamente irrompible en algo que se adivina en segundos. El análisis también mostró por qué el parámetro `configuracion` no aporta seguridad: no amplía el conjunto de valores posibles, solo redistribuye su probabilidad.

El intermediario demostró que la confidencialidad no implica autenticidad: RSA-OAEP protege el secreto en tránsito, pero no impide que alguien se haga pasar por el servidor sustituyendo su clave pública. Las firmas digitales cerraron ese hueco, pero solo porque la víctima ancla su confianza en una clave de verificación obtenida de antemano por un canal aparte. Si esa confianza se pudiera sustituir junto con todo lo demás, la defensa no serviría; por eso el verdadero punto de seguridad no es firmar, sino tener una raíz de confianza que el atacante no pueda tocar.

11. Fuentes

- Cuaderno Colab 1 del curso: *RSA, fundamentos, ataques y cifrado híbrido*, secciones 7 (RSA-OAEP) y 8 (aplicación cliente-servidor).
- Documentación de PyCryptodome (módulos `Crypto.Cipher.PKCS1_OAEP`, `Crypto.Cipher.AES` en modo GCM, `Crypto.Hash.HMAC`/`SHA256` y `Crypto.Signature.pss`).
- PKCS #1 v2.2 (RFC 8017) para RSA-OAEP y RSA-PSS.
- NIST SP 800-38D para el modo de operación AES-GCM.
- FIPS 198-1 para HMAC.